

Instruction set design

Instruction set design involves defining the set of operations, operands, and instruction formats that a processor will support. It determines how software communicates with hardware and influences performance, complexity, and efficiency of implementation.

Purpose of Instruction Set Design

1. To Define a Clear Interface Between Hardware and Software

The instruction set serves as the **boundary or interface** between hardware and software. It defines how software communicates with the processor, specifying what instructions the hardware can understand and how they are encoded.

2. To Determine What Operations the CPU Can Perform

Instruction set design decides **which operations** (like arithmetic, logic, control flow, and data movement) a processor will support and **how they will be encoded**.

3. To Make Programming Easier and Reduce Software Complexity

A well-designed instruction set helps programmers and compilers write **shorter, simpler, and more efficient** code. It provides a **regular and orthogonal structure**, meaning that all operations can use similar formats and addressing modes which reduces confusion and complexity.

4. To Optimize Performance and Power (Balancing Simplicity and Functionality)

The choice of instruction types, lengths, and addressing modes affects **processor performance, power consumption, and hardware complexity**.

1.2 Components of an Instruction

Opcode (Operation Code)

The **opcode** (operation code) is the most essential part of a machine instruction. It specifies the operation that the processor should perform, such as **ADD, LOAD, STORE, or JUMP**. The opcode acts as a signal to the control unit, directing the data path and ALU on what action to take. The number of bits allocated to the opcode determines how many unique instructions can be represented in the instruction set architecture (ISA). A fixed-length instruction format simplifies opcode decoding since the opcode always appears in a predictable position, which increases processing speed. For example, in RISC architectures like MIPS, the opcode usually occupies the first few bits of a 32-bit instruction.

The design of opcodes affects processor performance, instruction decoding complexity, and instruction compatibility between different CPU models.

Operands

The **Operands** are the data values or addresses that the opcode operates on. An operand can be a constant (immediate value), a register, or a memory location. Operands define “what” data will be used in performing the operation. The number of operands (zero, one, two, or three) depends on the instruction set design. For example, in a two-operand instruction ADD R1, R2, the CPU adds the contents of **R2** to **R1**. Operands can also differ based on architecture — in accumulator-based architectures, one operand is always the accumulator, while in register-based architectures, all operands are registers. The way operands are defined affects CPU design, memory usage, and the number of instructions required to perform a task.

Addressing Mode

The **addressing mode** specifies how to interpret or locate the operand referenced in an instruction. It tells the CPU whether the operand is stored directly inside the instruction, in a register, or at a memory address. The most common addressing modes are:

Immediate mode: Operand value is part of the instruction itself.

Register mode: Operand is in a processor register.

Direct mode: Instruction provides the memory address of the operand.

Indirect mode: Instruction specifies an address that points to the operand.

Indexed mode: Combines a base address and an offset for array access.

Addressing modes provide flexibility to handle constants, variables, arrays, and dynamic data structures. However, they also increase hardware complexity, as the CPU must calculate effective addresses before data can be accessed.

4. Instruction Format

The **instruction format** is the arrangement or layout of bits in a machine instruction. It determines how information is divided into fields such as opcode, operands, addressing mode bits, and immediate or displacement values. The choice of instruction format affects decoding simplicity, memory efficiency, and instruction execution speed.

There are mainly two types of formats:

Fixed-length format: All instructions are of equal length (e.g., 32 bits in MIPS). This makes hardware implementation simple and fast.

Variable-length format: Instructions differ in length, making code more compact but decoding more complex (used in CISC architectures like x86).

The instruction format directly influences the instruction set’s overall structure and how efficiently the processor can fetch and decode instructions.

Types of Instructions

In computer architecture, **instructions** are categorized based on the **function they perform** within a processor. Each category of instruction helps the CPU perform a specific type of task such as moving data, performing arithmetic operations, controlling program flow, handling input/output, or managing processor states. The five major types of instructions are described below.

1. Data Transfer or Data Movement Instructions

These instructions are used to **move data** from one place to another within the computer system. The transfer may occur between registers, between memory and registers, or between input/output devices and memory.

Purpose:

To copy data between processor components.

To load data from memory into registers or store results back to memory.

Common Examples:

LOAD – Transfers data from memory to a register.

STORE – Transfers data from a register to memory.

MOV – Moves data from one register to another.

PUSH/POP – Used for stack operations.

2. Arithmetic and Logical Instructions

These instructions are responsible for performing **mathematical and logical operations** on data. Arithmetic operations include addition, subtraction, multiplication, and division, while logical operations perform bitwise logic such as AND, OR, XOR, and NOT.

Purpose:

To process numerical or binary data inside the Arithmetic Logic Unit (ALU).

To perform comparisons and condition evaluations in programs.

Common Examples:

ADD – Adds two values.

SUB – Subtracts one value from another.

MUL/DIV – Multiplication or division of operands.

AND, OR, NOT, XOR – Logical operations for decision-making or masking bits.

Example:

```
ADD R1, R2      ; R1 ← R1 + R2
AND R3, R1, R2  ; R3 ← R1 AND R2
```

Explanation: These instructions modify the content of registers to perform computation or logical tests, forming the foundation of all processing tasks.

3. Control or Branch Instructions

These instructions are used to **alter the normal sequence of program execution**. Normally, instructions are executed sequentially, but branch instructions allow the program to jump to another address or subroutine.

Purpose:

To implement decision-making and loops in programs.

To handle subroutine calls and conditional branching.

Common Examples:

JMP / JUMP – Transfers control to another part of the program.

CALL / RET (RETURN) – Used to invoke and return from subroutines.

BRANCH (BEQ, BNE) – Conditional jumps based on comparison results.

Example:

```
CMP R1, R2      ; Compare R1 and R2
BEQ LABEL1      ; If equal, branch to LABEL1
```

Explanation: Control instructions help in creating structured logic like loops, function calls, and conditional statements.

Input/Output (I/O) Instructions

These instructions are designed for **communication between the CPU and peripheral devices** such as keyboards, printers, disks, or sensors. They allow the processor to send and receive data to/from external hardware.

Purpose:

To perform data transfer between I/O devices and processor memory.

To manage input and output operations efficiently.

Common Examples:

IN – Reads data from an input device into a register.

OUT – Sends data from a register to an output device.

5. Processor Control Instructions

Processor control instructions are used to **control the internal operation or state of the processor** itself. They do not operate on data but affect the processor's functioning, interrupt handling, or execution mode.

Purpose:

To start, stop, or control CPU operation.

To manage interrupt systems, halt operations, or perform no-operation tasks.

Common Examples:

HLT (Halt) – Stops CPU execution.

NOP (No Operation) – Performs no action but consumes one clock cycle.

INT (Interrupt) – Triggers or responds to interrupts.

WAIT – Pauses execution until a specific condition occurs.

Instruction Format and Encoding

The **Instruction Format** specifies **how bits are arranged within an instruction word** that is, how the CPU interprets the binary pattern to know what operation to perform, what data to use, and where to find it.

Each instruction in a computer's memory is represented in **binary (0s and 1s)**. The format of these bits determines how the processor **decodes and executes** the instruction.

Purpose of Instruction Format

To define **how instructions are represented in binary form**.

To provide the **layout** of different fields (opcode, operands, addressing information).

To ensure **efficient decoding** by the CPU.

To balance between **simplicity (hardware decoding)** and **richness (number of instruction types)**.

To **standardize** how processors interpret instructions so that compilers and hardware can communicate effectively.

1.5 Instruction Length & Complexity

Instruction length affects both processor speed and hardware complexity. **Short instructions** are easier to fetch, decode, and execute, leading to faster performance and simpler control logic a common feature in **RISC architectures**. However, **longer instructions** can hold more information such as larger addresses or multiple operands, allowing for more powerful operations as seen in **CISC designs**. The challenge for designers is to **balance simplicity and functionality** keeping instructions short enough for efficient decoding, yet long enough to support the required operations and addressing modes.

1.6 Number and Type of Operands

The number of operands in an instruction defines how many data items it can directly use.

Zero-address (Stack-based): Operands are taken from the stack automatically. Example: ADD adds the top two stack values.

One-address: One operand is specified, and the other is implied (usually the accumulator). Example: $\text{ADD } X \rightarrow \text{Acc} = \text{Acc} + X$.

Two-address: Both source and destination are given. Example: $\text{ADD } A, B \rightarrow A = A + B$.

Three-address: Two source and one destination operands are specified. Example: $\text{ADD } R1, R2, R3 \rightarrow R1 = R2 + R3$.

1.7 RISC vs CISC

The two main processor design philosophies are **RISC (Reduced Instruction Set Computer)** and **CISC (Complex Instruction Set Computer)**.

RISC architectures use a **small, simple, and fixed set of instructions**, each designed to execute very quickly often in a single clock cycle. Instructions are usually of **fixed length**, which simplifies decoding and allows for efficient pipelining. RISC focuses on performing simple operations rapidly, using more instructions for complex tasks. Common examples include **ARM, MIPS, and RISC-V**.

In contrast, **CISC architectures** have a **large and complex instruction set**, where a single instruction can perform multiple low-level operations such as memory access and arithmetic. These instructions are often of **variable length**, which can make decoding slower but reduces the number of lines of assembly code needed. CISC designs aim to make programming easier and code more compact. Common examples include **Intel x86 and AMD processors**.

1.8 Design Considerations

While designing an instruction set, architects consider:

Simplicity and regularity

Orthogonality (independent use of instructions and addressing modes)

Support for data types (integer, float, character)

Instruction frequency (how often each is used)

Ease of decoding and pipelining

Addressing Modes

2.1 Addressing Modes – Definition

In computer architecture, an **addressing mode** defines the **method used by the CPU to locate the operands** (data values) required for executing an instruction. In simple terms, it tells the processor **where to find the data** whether the data is stored in a **register**, a specific **memory location**, or is **directly included** within the instruction itself.

Different addressing modes provide flexibility and efficiency in instruction design. For example, some instructions might operate on data already present in a register, while others may need to fetch data from memory or use a constant value. By using multiple addressing modes, the same instruction set can handle a variety of programming tasks without requiring many different instructions.

Addressing modes are important because they directly influence the **efficiency, flexibility, and performance** of the processor. They help programmers and compilers optimize how instructions access data, minimizing memory operations and improving speed.

Common addressing modes include **Immediate, Register, Direct, Indirect, Indexed, and Relative addressing**, each with its own way of specifying where operands are located.

2.2 Common Addressing Modes

Mode	Description	Example
Immediate	Operand value is directly in the instruction.	MOV R1, #5
Register	Operand is in a register.	ADD R1, R2
Direct (Absolute)	Instruction gives the memory address directly.	MOV R1, 5000
Indirect	Address of operand is held in a register or memory.	MOV R1, (R2)

Mode	Description	Example	
Indexed	Effective address = Base address + Index value.	MOV 100(R2)	R1,

2.3 Importance of Addressing Modes

Provide **flexibility** in accessing data.

Enable **compact programs** (fewer instructions).

Help implement **loops, arrays, and pointers** efficiently.

2.4 Trade-offs in Addressing Mode Design

More modes → greater flexibility, but more decoding complexity.

Fewer modes → simpler and faster processor design (RISC style).

3. Relationship Between ISA, Instruction Set Design, and Addressing

ISA defines the interface between software and hardware.

Instruction Set Design specifies the structure and meaning of instructions.

Addressing Modes define how those instructions access data.

Together, they determine **how efficiently a processor executes programs**.

Note: The above notes were self-prepared using authentic sources including Hennessy & Patterson (Computer Architecture: A Quantitative Approach, 6th Edition) and William Stallings (Computer Organization and Architecture, 11th Edition).